

B U P T

密码工程

第13讲、PKI的现实问题、PKI的简单实现

授课教师：王晨宇

时间：2026年6月14日

5.6

PKI的现实问题

► PKI的现实问题

B U P T



✓ 5.6.1 前言

✓ 5.6.2 身份

✓ 5.6.3 认证权限

✓ 5.6.4 信任

✓ 5.6.5 间接授权

✓ 5.6.6 直接授权

✓ 5.6.7 信用系统

✓ 5.6.8 撤销

✓ 5.6.9 PKI的优劣势

✓ 5.6.10 如何选择密钥管理方案



理想与现实的差距

- PKI（公钥基础设施）在理论上设计得非常优雅，拥有完整的密钥管理与信任体系。然而，现实并不如设想般理想。
- 实际部署中的PKI系统常因复杂性、信任链断裂、密钥管理成本等问题而难以落地。
- 多年来的宣传虽然塑造了PKI的理想形象，但忽视了与实际应用场景的结合。
- 理论完美，实践困难，正是PKI现实困境的真实写照。



身份 ≠ 名字

- 在现实世界中，我们常以“名字”来指代一个人，但名字远非唯一标识。
- 在一个班级里叫“李伟”的可能有3个，社交网络上叫“Sky”的可能有上万。
- 真正的“身份”是用来区分一个具体的人的，这种区分需要超越口头或文本的名字。
- 在一个小群体中，一个名字就足够，但在人口密集或线上环境中，名字的歧义会迅速显现。



身份识别的挑战

- 在数字世界中，你的“身份”常常是一个用户名、邮箱或手机号。
- 问题是，这些标识并不是全球唯一且不可篡改的。
- 例如：
 - 同一个手机号可能会在你停用后转给别人。
 - 多人共享一个外卖账号或Netflix账户。
 - 某人冒用你的昵称注册社交账号，甚至申请贷款。
- 身份不等于账户，身份也不等于名字。
- 身份要解决的是：你是谁，以及你说这句话的可信度。



PKI视角下的身份问题

- 公钥基础设施（PKI）试图用公钥来标识一个“身份”：Alice 的公钥就代表她自己。
- 但在实践中，我们面临的问题是：谁来告诉我这个公钥就属于 Alice？
 - 这就引出了认证、信任链、证书颁发等复杂机制。
- 身份识别是技术、社会规范与信任关系的交集。一个好的身份体系，必须考虑唯一性、稳定性、可验证性。



谁来决定“你是谁”？

- 在一个PKI系统中，不是每个人都能随意给别人发证书。
- 一个核心问题是：谁有权限分配名字？谁有权限颁发密钥？
- 比如：
 - 公司知道某人是自己的员工，所以有权为其颁发公司内部VPN证书。
 - 银行知道某人是自己的客户，所以有权绑定其真实姓名并发放数字证书。
- 这背后的隐含逻辑是：组织内部有权定义自己“信任的人是谁”。



命名权限 ≠ 命名控制(1)

- 公司有权定义谁是员工，却无法控制这个人的“法律实名”或“社交媒体ID”。
- 比如：
 - 公司给员工“小李”发的邮箱是 li@company.com，即便他身份证叫“李大伟”，这并不冲突。



命名权限 ≠ 命名控制(2)

□在PKI中也是类似：

□某个CA（证书颁发机构）可以决定 bob@cloudservice.com 属于谁，但它不一定能证明“Bob”这个名字在全球唯一。

□每个组织只能在自己的命名空间内建立可信关系。

□所以设计PKI时必须明确：**每个名字是谁授权的、在哪个范围内有效。**



PKI的核心是信任

- PKI解决的是“我怎么知道你是谁”的问题。
- 核心角色 CA被赋予巨大权力，一旦CA签发了错误的证书，整个系统的安全就可能被攻破。
- 举例：
 - 如果CA泄露密钥，攻击者可仿冒一切下游身份，造成灾难性后果。



CA ≠ 全球信任中心

- 很多机构以为PKI就等于“全球CA说了算”，但在现实世界中，没有任何一个组织能赢得所有人的信任。
- CA像大楼的“门卫”，只验证身份，却无法判断来访者是否值得信任。
- 更现实的方式是“本地自信任”：
 - 银行为自己的客户发证书，不依赖外部CA。
 - 公司为自己的员工发VPN证书，自建私有CA。



信任的本质是“契约”

- 真正安全的信任关系，不是因为“权威”，而是因为“协议”：
 - 你发的证书我认，因为我们事先说好了。
- 这也是现代密码系统设计的关键原则：
 - 默认不信任，显式建立信任。
 - 不是“谁都能签证书”，而是“我信谁签的证书”。
- PKI不是万能网络护盾，而是一种灵活的信任协商框架。



什么是“间接授权”？

- 在传统的PKI设计中，授权信息依附于“名字”：
- 系统将“名字”与密钥绑定，用“名字”来控制访问权限。
- 例如：
 - 你是 Bob，只要你的名字在 ACL（访问控制列表）里，你就可以访问 /usr/bob 目录。
- 这种方式叫“间接授权”：
 - 不是针对密钥本身授权，而是授权给一个“名字”。



间接授权的问题

□ 问题1：名字不唯一 → 容易混淆。

□ ACL里的 Bob 和证书里的 Bob 是同一个人吗？

□ 问题2：绑定链条长 → 攻击面大。

□ 从密钥 → 名字 → ACL → 权限，链条中的任何一环被篡改都可能引发安全问题。

□ 问题3：难以跨系统。

□ 不同组织对“Bob”的定义不一致，授权难以共享。



什么是“直接授权”？

- 直接授权的核心思想是：用户的证书直接绑定其可执行的操作，而非间接通过名字与ACL判断权限。
- 系统直接验证“该证书是否拥有操作X的权限”，无需再确认是谁。
- 例如：
 - 一个VPN接入控制系统可以直接验证证书中是否包含“访问公司内网”的权限，无需知道该用户是谁。
- 这种方式更加直接、清晰，适合大规模、跨组织、自动化的权限判定场景。



直接授权的优势

□减少依赖链条，提升系统健壮性：

- 去除了对名字、ACL、身份系统的依赖，证书中直接声明权限。避免名字冲突、跨域不一致、中心数据库故障等问题。

□简化管理与配置：

- 权限的变更只需更新证书，无需修改中心数据库。

□提升安全性：

- ACL数据库常是攻击目标，而直接授权将权限分散在证书中，降低单点风险。



证书中还需要名字吗？

- 虽然权限判断不再依赖名字，但证书中仍然可以包含标识信息，如名字、邮箱、用户ID等。主要用于审计、日志分析、追责等非实时目的：
 - 例如，一笔“J.Smith”发起的支付日志，可通过证书确认其密钥确实拥有该操作权限。
 - 银行通过名字可知道是谁完成了转账，VPN日志可追踪访问记录。
- 授权靠密钥：系统验证是否有权执行操作；
- 审计靠名字：用于记录“是谁”执行了该操作。



什么是“信用系统”

- 信用系统是指基于数字签名证书构建的权限传递机制。
- 用户可以通过签发带有权限信息的证书，将访问权限委托给他人。
- 例如：
 - 如果 Alice 拥有文件 X 的读权限，她可以签署一张证书授权 Bob 也能读取该文件。
 - Bob 若想访问 X，需要向系统出示这张证书，系统验证通过后即可放行。



权限链与有效期

- 信用系统支持“链式授权”，如 Alice → Bob → Charlie。
- 授权证书中可以包含有效期或使用限制等约束条件：
- 例如：
 - “允许访问文件 X 的时间为1小时内”。
 - “不允许继续转发授权”。
- 这些限制信息需写入证书并被验证机制识别，才能确保安全可控。



信用系统为何很少使用?

- 理论上它具备高效与灵活性，但实际部署较少，主要原因包括：
 - 系统复杂，用户难以理解与管理授权链。
 - 授权边界模糊，容易引发权限滥用或过度授权。
 - 授权语义难以统一，编写表达权限的“授权语言”难度大。



细粒度访问控制的困境

- 信用系统常诱导管理员进行“过度细化”的权限分配：
 - 例如按项目、时间、组别精细划分权限。
 - 最终导致用户忘记自己是否拥有权限，甚至访问失败。
- 某些单位试图让用户自助选择权限范围，结果出现混乱或滥用。
- 当前几乎无系统能在权限描述上做到安全与可用的平衡。



证书层级结构与权力分布

- 在CA体系下，信用系统需明确子CA是否具备发放权限证书资格。
- 若中心CA签发的证书中未限制子CA用途，等于赋予其“无限权力”。
- 因此：
 - 每层CA的权限应在证书中精确标注。
 - 子CA不应随意发放访问权限。
- 否则将破坏整个系统的最小授权原则。



证书撤销的挑战

- 在PKI中，撤销是最难的问题之一。
- 一旦证书被盗用、泄露或失效，必须撤销该证书。
- 例如：
 - Bob的计算机被入侵，攻击者复制了其私钥。
 - Alice被调岗或离职等情况，都需要及时撤销对应证书。
- 撤销困难的主要原因：
 - 证书传播广泛，用户难以“撤回”。
 - 用户可能根本忘记了该证书仍然有效。



撤销需要系统性地解决以下三大变量

□撤销速度：

- 从发出撤销指令到生效的最大延迟？

□撤销可靠性：

- 哪些情况下撤销可能“失败”？如何接受一定风险？

□撤销处理能力：

- 系统是否可支持高并发的撤销请求？



撤销列表 (CRL)

- CRL (Certificate Revocation List) 是一种集中维护撤销证书的数据库。
- 验证者在使用证书前，必须检查该证书是否在CRL中。
- CRL的优点：
 - 可靠性高，一旦证书在CRL中，即刻视为无效。
 - 没有撤销数量限制，理论上可无限追加。
- CRL的挑战：
 - 所有用户必须在线访问CRL数据库。
 - CRL数据库本身成了高优先级攻击目标，若失效会导致严重后果。



CRL部署的典型策略

- 分布式部署多个CRL副本，提高可用性与容灾能力。
- 典型做法：定期从主服务器下载CRL副本至本地缓存。
- 限制与代价：
 - 分布式CRL的构建与运维成本极高。
 - 很多系统将CRL作为“附带数据”下发，增加传输负担。
- 实践中常见的妥协：
 - 设置合理的本地CRL缓存更新频率（如每半小时）。
 - 减少CRL文件大小，每设备仅存储部分最关键撤销数据。



短的有效期：替代撤销机制的新方式

- 利用现有证书机制，仅签发短有效期证书（如10分钟~24小时）。
- Alice 想使用证书时，从 CA 获取一个新证书，只要该证书有效，就能使用。
- 不需要独立的 CRL 数据库，简化系统架构，降低管理负担。
- 问题：
 - 挑战有效期不能太短（如少于10分钟）。
 - 用户仍需持续在线才能获取新证书。



为什么不能忽略撤销?

- 密钥被盗用、权限变更等情境下都需要撤销。
- 没有撤销功能的 PKI 等于“纸上系统”。
- 例如：
 - Bruce十年前泄露了PGP密钥，邮件仍然用其加密。
 - 如果Bob的私钥被窃取，攻击者可伪造相应证书副本。



- PKI（公钥基础设施）的设计初衷是让通信双方（如 Alice 和 Bob）可以无需第三方参与，就能建立安全的加密通道，实现可信通信。
- 例如：
 - 面对无法依赖在线服务的情况，PKI 提供了“离线验证”能力。
- 然而，这一优势建立在完整的撤销体系基础上。如果没有撤销机制，PKI 就无法提供真正的安全保证。
- 在实践中，撤销要求所有用户保持在线，以便随时验证证书是否有效。



与密钥服务器的比较

□ 另一种方案是使用**中心化密钥服务器**：

□ 所有用户通过它来交换和验证密钥，前提是大家都在线。

□ 如果中心服务器宕机，通信就会中断。

□ 这种模式对实时可用性要求较高，抗故障能力较弱。

□ 相比之下，PKI 允许证书提前发放，同时支持可验证签名、不可否认性以及跨域信任体系，这是密钥服务器方案难以提供的。



PKI 的缺点

- PKI 的缺点也很明显，维护复杂、成本较高，涉及撤销系统、分层 CA 管理、公钥计算等多个复杂环节。
- 在理论上，PKI 提供了安全性和灵活性上的理想模型，但落地部署时仍需大量工作。
- 关键问题是如何确保 CA 密钥不被泄露，以及如何可靠地传递撤销信息。



小系统 vs 大系统，怎么选

□ PKI 的复杂性对小系统不友好，其优势在于支持大规模分布。

□ 小型系统：

□ 用密钥服务器更简单，维护方便，适合规模不大的单位。

□ 大型系统：

□ 更适合用 PKI，它支持多人协同、跨区域使用。

□ 可通过限制子 CA 的权限，实现灵活的信任扩展，扩展性更强。



1. PKI系统依赖于证书颁发机构(CA)的权威和安全，假如CA遭遇安全事件或被攻击，整个生态系统该如何自我修复和恢复信任？

5.7

PKI的简单实现

► PKI的简单实现



- ✓ 5.7.1 数字证书
- ✓ 5.7.2 OpenSSL
- ✓ 5.7.3 PKI的代码实现



什么是数字证书

- 数字证书遵循X.509标准，规定了证书及证书撤销列表的格式。
- X.509标准为证书的生成、验证和管理提供了统一规范。
- 证书用于证明公钥的归属及持有者身份，保证网络通信的安全性。



证书数据结构 (1)

- 证书包含多个数据域，关键字段包括：
- 版本号：指明证书采用的X.509版本。
- 序列号：由CA分配的唯一数字标识，用于标识和撤销证书。
- 签名算法标识：说明证书签名时采用的算法。
- 签发机构：颁发证书的CA名称，包含国家、省市、组织机构等信息。
- 有效期：证书的生效时间和失效时间，每次使用证书时需检查其是否在有效期内。



证书数据结构 (2)

- **主题名称**：证书持有者的唯一名称，包括单位、组织及个人信息，可能包含邮箱地址。
- **持有者公钥信息**：包含持有者的公钥及其使用的算法。
- **签发者唯一标识符**：用于区别拥有相同名称的不同CA。
- **持有者唯一标识符**：用于区别拥有相同名称的不同证书持有者。
- **CA签名值**：证书签发机构对证书内容的数字签名，确保证书未被篡改。

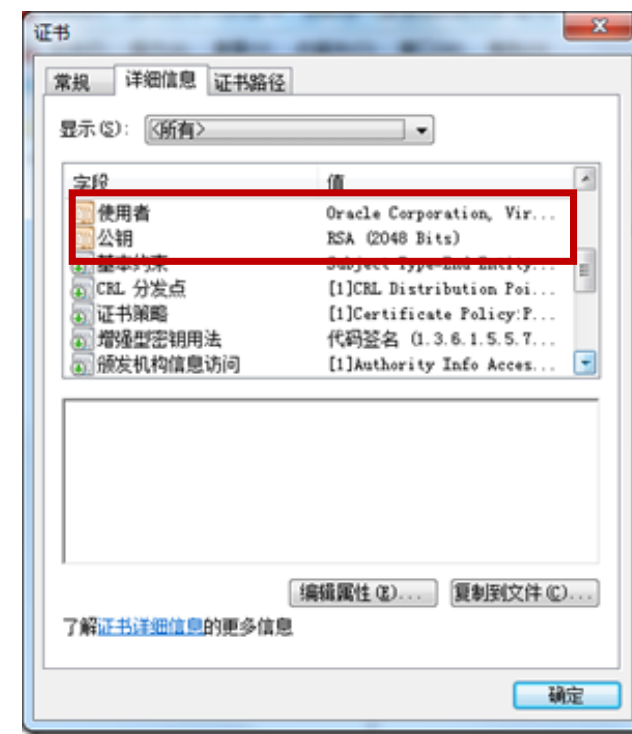
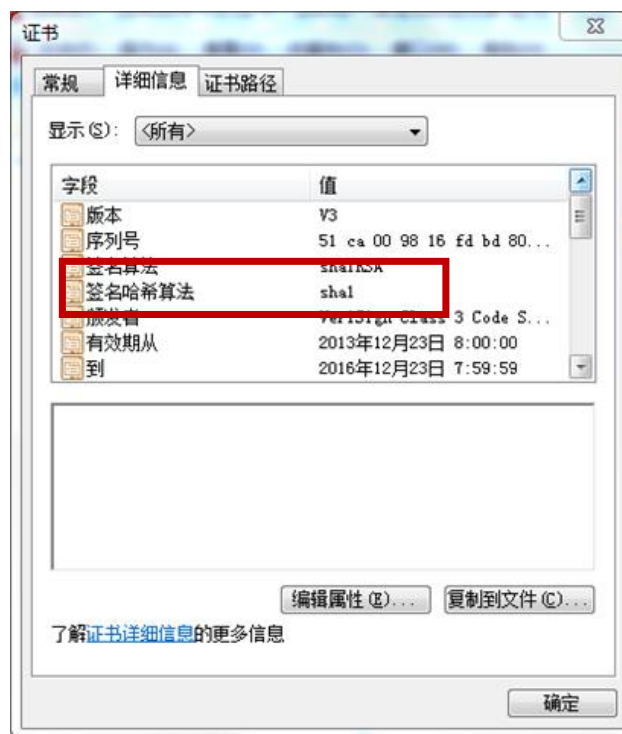
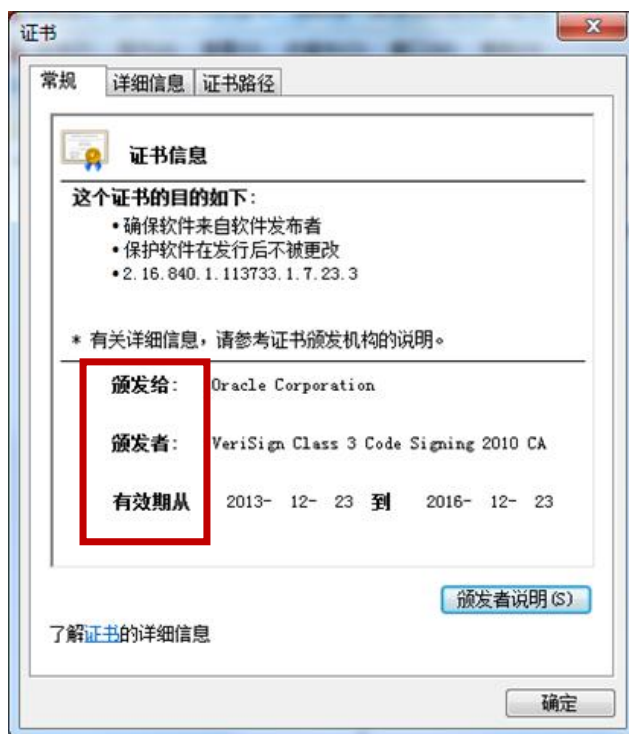


PKI的简单实现

5.7.1 数字证书(4)

B U P U T

证书格式举例





数字证书的常见格式

□ X.509证书的编码格式主要有两种：

□ **DER编码（ASCII）**：对应文件后缀为 .der、.cer、.crt。

□ **PEM编码（Base64）**：对应文件后缀为 .pem、.cer、.crt。

□ 不同编码格式适用环境不同，例如：

□ Firefox浏览器支持DER编码的证书，即ASCII格式的证书。



背景介绍

- OpenSSL 是一个广泛使用的开源实现，支持SSL和TLS协议。
- SSL/TLS（Secure Socket Layer / Transport Layer Security）是基于PKI实现网络安全通信的协议。
- OpenSSL提供了丰富的密码算法库和应用程序，支持构建完整的PKI框架。



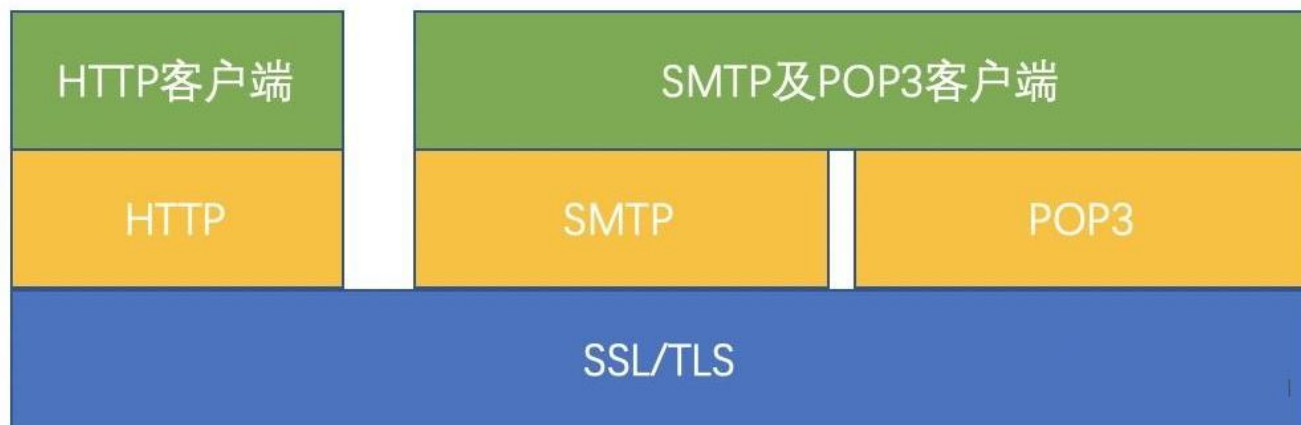
什么是SSL/TLS协议

- SSL/TLS是一种密码通信框架，是世界上使用最广泛的密码通信方法。
- SSL/TLS综合运用了密码学中的对称密码，消息认证码，公钥密码，数字签名，伪随机数生成器等，可以说是密码学中的集大成者。
- **SSL(Secure Socket Layer) :**
 - 是1994年由Netscape公司设计的一套协议，并与1995年发布了3.0版本。
- **TLS(Transport Layer Security) :**
 - 是IETF在SSL3.0基础上设计的协议，实际上相当于SSL的后续版本。



SSL/TLS 协议在网络协议栈中的位置

- SSL/TLS协议位于传输层和应用层之间，主要负责通过握手协议建立安全连接。
- 通过记录协议实现数据的加密传输与完整性保护，保障HTTP、SMTP、POP3等应用协议的安全通信。



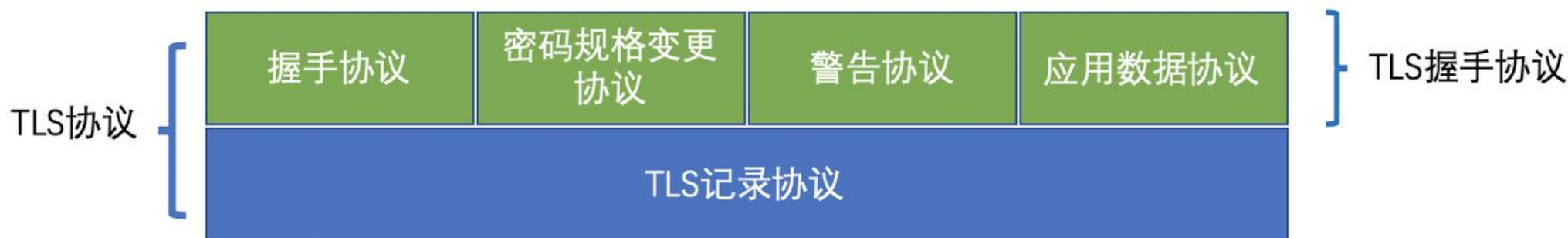


TLS协议结构与主要功能

□ TLS协议分为两层：

□ 底层为 TLS记录协议，负责使用对称加密对传输消息进行加密保护。

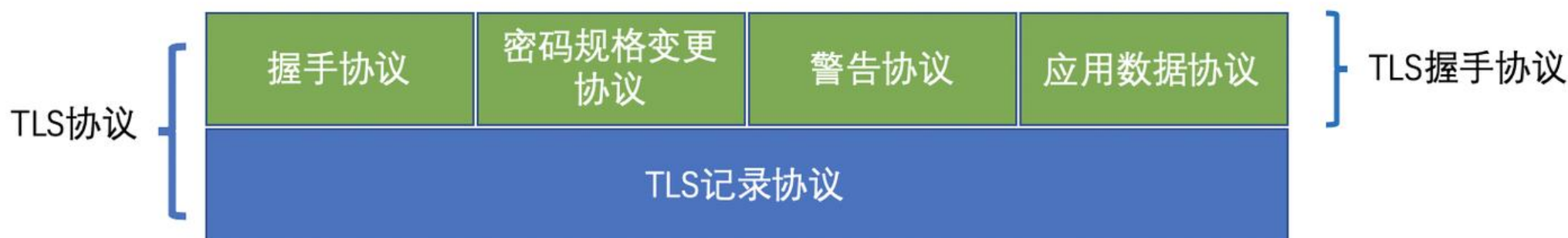
□ 上层为 TLS握手协议，包含4个主要部分：**握手协议、密码规格变更协议、警告协议、应用数据协议。**





TLS协议结构与主要功能

- 握手协议：负责客户端和服务端协商密码算法与共享密钥，包括证书认证，是最复杂的部分。
- 密码规格变更协议：负责传达密码算法切换的信号。
- 警告协议：负责在发生错误时向对方发送警告信息。
- 应用数据协议：负责传输TLS承载的应用层数据。





OpenSSL

- OpenSSL是SSL/GmSSL协议的开源实现。
- 它由3部分组成：
 - 密码算法库
 - SSL/TLS协议
 - 应用程序



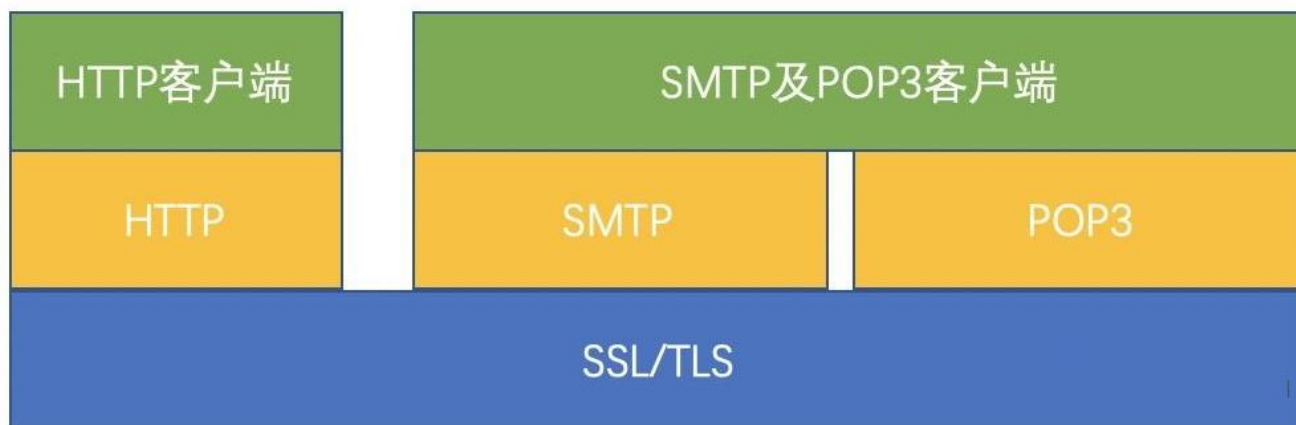
密码算法库

- 提供现代密码学中主流的密码算法和标准支持，包括：
 - 公钥算法（非对称）：如RSA、DSA、EC、SM2等。
 - 对称加密算法：如AES、DES、SM4、RC4等。
 - 信息摘要算法（MAC）：如MD5、SHA系列、SM3等。
 - 支持X.509数字证书标准、OCSP在线证书状态查询、CRL证书撤销列表等协议。



SSL/TLS协议实现

- OpenSSL实现了SSL协议的多个版本（V1、V2、V3）及TLS协议1.0。
- 基于密码算法库构建，负责网络通信中的安全连接建立与维护



[SSL详细流程](#)



应用程序集合

- 提供丰富的实用工具和程序，主要包括：
 - 多种算法的加密程序和密钥生成程序（如RSA、MD5）
 - 证书签发和验证程序（如CA管理、X.509证书处理、CRL列表）
 - SSL连接测试程序
 - 其他标准安全应用



搭建CA - 工作环境初始化

□ 根目录通常位于 /etc/pki/CA。

□ 需要创建以下基础文件和目录：

□ index.txt：证书数据库索引文件

□ serial：存储当前证书序列号

□ csr目录：用于存放证书签名请求文件

□ crlnumber：存储CRL序列号这些文件和目录只需首次初始化时创建并赋初值。

```
# touch /etc/pki/CA/{index.txt,serial} #这些文件配置文件中提到
# echo "01" > /etc/pki/CA/serial #只有第一次这么做，为证书生成编号
# mkdir csr #建立此目录准备放置签署证书请求的
# echo 00 > /etc/pki/CA/crlnumber #只有第一次这么做，为证书吊销列表生成编号
```



搭建CA - 生成私钥

- 私钥文件非常重要，除管理员外，其他用户和组均不应拥有访问权限。
- 权限建议设置为 600 或 400，防止私钥泄露。
- 生成示例命令（RSA 2048位密钥）：

```
cd /etc/pki/CA  
umask 077  
openssl genrsa -out private/cakey.pem 2048
```



搭建CA - 配置CA的签发机构信息

□配置信息主要包括：

□国家代码、省/州、市、组织名、部门、通用名（CN）、电子邮件地址等。

□配置文件一般为 openssl.cnf，其中填写上述信息，供证书签发时使用。



搭建CA - CA_default配置

- ❑ 定义了CA的基本路径和文件位置，确保所有证书和证书撤销列表（CRL）等文件都存放在统一目录。
- ❑ 重要参数包括：
- ❑ **dir**：CA根目录certs。
- ❑ **crl_dir**：证书和CRL存放路径。
- ❑ **database**：证书数据库索引文件。
- ❑ **serial**、**crlnumber**：证书和CRL当前序列号。
- ❑ **private_key**：私钥文件路径。
- ❑ 通过这些路径配置，OpenSSL可以正确管理CA文件和流程。

```
[ CA_default ]

dir = /etc/pki/CA # Where everything is kept
certs = $dir/certs # Where the issued certs are kept
crl_dir = $dir/crl # Where the issued crl are kept
database = $dir/index.txt # database index file.
#unique_subject = no # Set to 'no' to allow creation of
# several certificates with same subject.
new_certs_dir = $dir/newcerts # default place for new certs.

certificate = $dir/cacert.pem # The CA certificate
serial = $dir/serial # The current serial number
crlnumber = $dir/crlnumber # the current crl number
# must be commented out to leave a V1 CRL
crl = $dir/crl.pem # The current CRL
private_key = $dir/private/cakey.pem # The private key
RANDFILE = $dir/private/.rand # private random number file

x509_extensions = usr_cert # The extensions to add to the cert

# Comment out the following two lines for the "traditional"
# (and highly broken) format.
name_opt = ca_default # Subject Name options
cert_opt = ca_default # Certificate field options
```



搭建CA - policy_match 策略匹配规则

□ 该策略用于匹配证书请求中的域信息，决定哪些字段必须匹配、哪些为可选。

□ 常用字段包括：

- countryName（国家代码）
- stateOrProvinceName（省/州名）
- organizationName（组织名称）
- organizationalUnitName（部门，可选）
- commonName（通用名，必填）
- emailAddress（电子邮件，可选）

```
policy = policy_match #使用policy_match来匹配请求的域信息
[ policy_match ]
countryName = match
stateOrProvinceName = match
organizationName = match
organizationalUnitName = optional
commonName = supplied
emailAddress = optional
```

□ 通过策略配置，确保证书签发时符合预设规则。



搭建CA - 证书请求中的约束

- 该部分设置了证书请求中的默认字段和约束，方便自动化签发和统一规范。
- 配置示例包括：
- 国家代码默认为CN（中国），长度限制为2。
- 省份、市区、组织、部门等信息可设默认值。
- 通用名（Common Name）最长64字符。
- 电子邮件地址最长64字符。
- 通过默认值配置，简化证书请求填写流程，提高一致性。

```
default_days = 3650 #证书期限

[ req_distinguished_name ]

#默认机构信息修改
countryName = Country Name (2 letter code)
countryName_default = CN
countryName_min = 2
countryName_max = 2

stateOrProvinceName = State or Province Name (full name)
stateOrProvinceName_default = JS

localityName = Locality Name (eg, city)
localityName_default = NJ

0.organizationName = Organization Name (eg, company)
0.organizationName_default = NJU

# we can do this but it is not needed normally :-
#1.organizationName = Second Organization Name (eg, company)
#1.organizationName_default = World Wide Web Pty Ltd

organizationalUnitName = Organizational Unit Name (eg, section)
organizationalUnitName_default = Heyuan

commonName = Common Name (eg, your name or your server's hostname)
commonName_max = 64

emailAddress = Email Address
emailAddress_max = 64
```




搭建CA - 证书请求交互示例

- 该部分通过openssl req命令生成证书请求时的交互过程。
- 用户需依次输入：国家代码、省份、城市组织、部门通用名（一般为服务器域名）邮箱地址
- 也可直接回车采用默认值，或输入.留空。
- 交互式填写帮助确保请求内容准确，符合CA策略。

```
# openssl req -new -x509 -key private/cakey.pem -out cacert.pem -days 3650
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [CN]:
State or Province Name (full name) [JS]:
Locality Name (eg, city) [NJ]:
Organization Name (eg, company) [NJU]:
Organizational Unit Name (eg, section) [Heyuan]:
Common Name (eg, your name or your server's hostname) []:www.heyuan.com
Email Address []:webmaster@heyuan.com
```

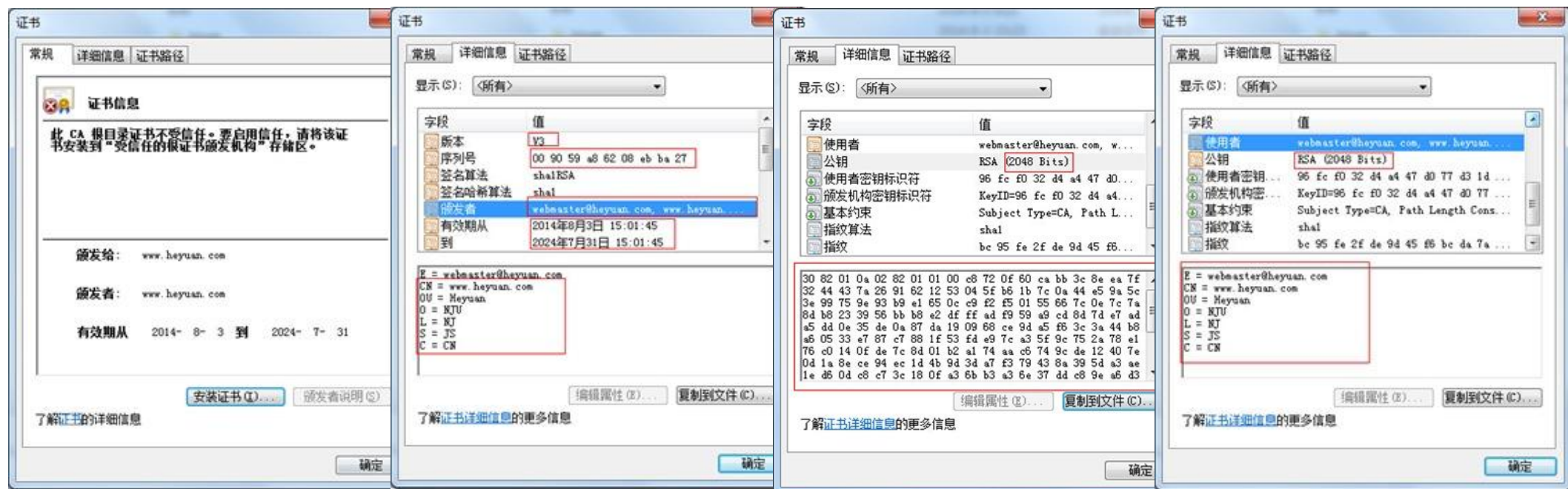


PKI的简单实现 5.7.3 PKI的代码实现(8)

B U P T

搭建CA - 自签署证书

□ 经过前面的步骤，在 /etc/pki/CA 目录下已经生成了CA的自签署证书。这样，一个简单的CA环境就搭建完成，具备了签发证书的能力。





节点申请证书 - 目录和密钥生成

- 创建SSL存放目录 /etc/httpd/ssl，为存储证书和密钥准备空间。
- 使用openssl genrsa命令生成2048位RSA私钥文件 httpd.key，确保密钥安全。

```
# mkdir /etc/httpd/ssl  
# (umask 077;openssl genrsa -out /etc/httpd/ssl/httpd.key 2048)
```



节点申请证书 - 生成证书请求

- 交互式填写请求信息，包括国家、省市、组织、通用名（服务器域名）及邮箱。
- 这些信息将包含在证书中，用于身份识别。

```
[root@localhost ~]# openssl req -new -key /etc/httpd/ssl/httpd.key -out /etc/httpd/ssl/httpd.csr
```

```
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [XX]:CN
State or Province Name (full name) []:JS
Locality Name (eg, city) [Default City]:NJ
Organization Name (eg, company) [Default Company Ltd]:NJU
Organizational Unit Name (eg, section) []:Heyuan
Common Name (eg, your name or your server's hostname) []:www1.heyuan.com
Email Address []:web1@heyuan.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```



节点申请证书 - 提交给CA

□使用scp命令将生成的CSR文件从服务器复制到CA目录，方便后续证书签发。

```
# scp /etc/httpd/ssl/httpd.csr 192.168.60.172:/etc/pki/CA/csr/
```



CA签署证书

- 通过CA使用openssl ca命令对CSR进行签发，生成服务器证书 httpd.crt，指定有效期（如1000天）。
- 查看索引文件 index.txt，确认证书状态和信息。
- 证书文件及相关信息将保存在指定目录，准备在服务器部署使用。

```
# openssl ca -in /etc/pki/CA/csr/httpd.csr -out /etc/pki/CA/csr/httpd.crt -days 1000
# cat index.txt
V 170429085617Z 01 unknown /C=CN/ST=JS/O=NJU/OU=Heyuan/CN=www1.heyuan.com/emailAddress=web1@heyuan.com#
cat serial
02
# ls /etc/pki/CA/newcerts/ /etc/pki/CA/csr/
/etc/pki/CA/csr/:
httpd.crt httpd.csr

/etc/pki/CA/newcerts/:
01.pem
```



证书撤销 - 查看证书信息

□使用openssl x509命令查看已签发证书的序列号和持有者信息，确认证书详情。

```
# openssl x509 -in /etc/httpd/ssl/httpd.crt -noout -serial -subject  
serial=01  
subject= /C=CN/ST=JS/O=NJU/OU=Heyuan/CN=www1.heyuan.com/emailAddress=web1@heyuan.com
```



证书撤销 - 验证证书状态

□ 查看CA的index.txt文件，确认证书当前的状态（如unknown表示未撤销）。

```
# cat /etc/pki/CA/index.txt  
V 170429085617Z 01 unknown /C=CN/ST=JS/O=NJU/OU=Heyuan/CN=www1.heyuan.com/emailAddress=web1@heyuan.com
```




证书撤销 - CA撤销证书

- 使用openssl ca -revoke命令撤销指定证书文件。
- 撤销后，查看index.txt，证书状态变为R（Revoked，已撤销）。

```
# openssl ca -revoke /etc/pki/CA/newcerts/01.pem
# cat index.txt
R 170429085617Z 140803091048Z 01 unknown /C=CN/ST=JS/O=NJU/OU=Heyuan/CN=www1.heyuan.com/emailAddress=web1@heyuan.com
```



证书撤销 - 更新证书撤销列表

□重置CRL序列号后，使用openssl ca -gencrl命令生成最新的证书撤销列表文件ca.crl。

```
# echo 00 > /etc/pki/CA/crlnumber
# openssl ca -gencrl -out /etc/pki/CA/crl/ca.crl
Using configuration from /etc/pki/tls/openssl.cnf
```



证书撤销 - 查看证书撤销列表

- 使用openssl crl命令查看生成的CRL内容，包含撤销证书的序列号和撤销时间。
- CRL文件由CA签名，供客户端验证证书有效性时参考。

```
# openssl crl -in /etc/pki/CA/crl/ca.crl -text -noout
Certificate Revocation List (CRL):
Version 2 (0x1)
Signature Algorithm: sha1WithRSAEncryption
Issuer: /C=CN/ST=JS/L=NJ/O=NJU/OU=Heyuan/CN=www.heyuan.com/emailAddress=webmaster@heyuan.com
Last Update: Aug 3 09:20:05 2014 GMT
Next Update: Sep 2 09:20:05 2014 GMT
CRL extensions:
X509v3 CRL Number:
0
Revoked Certificates:
Serial Number: 01
Revocation Date: Aug 3 09:10:48 2014 GMT
Signature Algorithm: sha1WithRSAEncryption
85:e5:7b:ad:7f:9c:04:03:2e:e8:cf:66:ac:e8:46:70:c3:9a:
8e:80:a6:8b:6a:39:31:cl:27:7a:28:e7:50:ba:8b:fd:07:f5:
c4:40:29:f2:e3:91:c7:e3:86:69:49:ad:28:cf:17:d1:a2:4c:
5d:52:f8:18:a2:84:32:3e:49:ad:04:54:3b:50:79:f8:e6:74:
a0:1a:85:40:e7:ce:c0:10:9c:c8:9e:19:f6:18:2d:5a:9a:85:
f8:2f:c8:75:56:4b:d9:15:39:f3:75:27:82:cc:04:02:09:be:
6e:10:6d:e9:bd:99:82:36:6e:f7:f7:c6:90:a7:18:66:2b:4e:
c2:ee:1c:0d:6e:2f:22:9f:da:d6:ac:e3:ae:e2:6e:50:64:43:
45:87:2c:63:18:de:c5:9e:d9:71:ff:84:9c:72:8f:20:2a:f4:
92:f6:2d:42:33:ae:f3:93:89:2f:57:b1:62:28:ba:c0:ec:31:
d8:b0:98:03:1f:73:fb:f5:91:52:52:98:ed:c0:30:a3:bb:8d:
5d:d6:54:13:e1:6a:6a:70:ef:c6:89:c9:4b:47:b0:59:0c:94:
8d:1b:9b:e4:09:e1:48:47:8b:4b:2c:5a:98:f0:4c:91:1f:65:
c8:76:49:13:d0:8f:79:9c:57:9b:7c:9d:35:d2:90:66:60:c3:
9e:95:7c:99
```



2.当系统规模扩大后，证书撤销列表（CRL）会变得庞大且难以维护，你认为有哪些新技术或方法可以替代传统CRL，实现高效的证书撤销管理？

谢谢大家
